

Subject: CS336 Assignment 1, Embedding Module

Date: from 2026/7/24 to 2026/7/25

Contents

Embedding Module	1
-------------------------	----------

Embedding Module

PyTorch 基础组件

在实现 Embedding 层之前，需要先理解 PyTorch 提供的“基础设施”。

nn.Module: 模型基类. 它是所有神经网络模块的母体；继承它之后，PyTorch 会自动追踪你定义的所有子模块与参数，并提供核心能力：

- `.to(device)`: 一键把模型搬到 GPU；
- `.train()` / `.eval()`: 切换训练 / 推理模式。

nn.Parameter: 被标记的张量. 它在本质上是特殊的 Tensor；一旦把一个 Tensor 定义为 `nn.Parameter`，它就会被自动加入模型的 `parameter()` 迭代器。只有被标记为 `Parameter` 的张量，优化器才能看到并更新它的梯度。

张量类型	更新梯度	随模型保存	随模型搬设备
<code>nn.Parameter</code>	是	是	是
<code>register_buffer</code>	否	是	是
普通 Tensor	否	否	否

选择依据：需要训练就用 `nn.Parameter`；需要随模型保存/搬运但不训练就用 `register_buffer`；临时中间变量用普通 Tensor。

torch.empty: 内存分配. 与 `torch.zeros`、`torch.ones` 不同，`empty` 只申请内存空间、不做数据填充，此时矩阵里是内存中的残留值（garbage values）。工程意义：在随后立即手动初始化的情况下，`empty` 比 `zeros` 更高效。

Embedding 的实现

1. 定义权重矩阵. 使用 `nn.Parameter` 初始化一个形状为 `[vocab_size, d_model]` 的权重矩阵；
2. 参数初始化（**Truncated Normal**）. 均值 $\mu = 0$ ，标准差 $\sigma = 1.0$ ，截断范围 `[-3, 3]`（即正负 3 倍标准差）。目的是防止初始值过大导致模型在训练初期失去稳定性；
3. 前向传播（**Forward**）. 直接利用 PyTorch 的索引语法 `self.weight[token_ids]`，按 ID 取出对应的行向量。

```
self.weight = nn.Parameter(torch.empty(vocab_size, d_model))
nn.init.trunc_normal_(self.weight, mean=0.0, std=1.0, a=-3.0, b=3.0)
return self.weight[token_ids]
```